Do We Need the Shadow?

Freezing-Aware Weight Versioning for Pipeline-Parallel Training: Findings

Animesh Jain, Rupankar Podder

Final presentation · Systems Engg. for Deep Learning (CS6886)

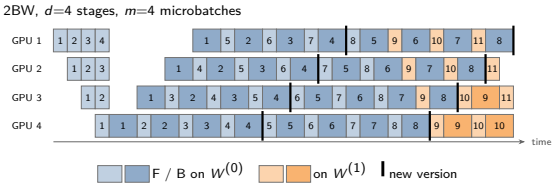
140+ training runs · 3 tracks · ResNet-56/110/50 and BERT Mini → Large · 2 × RTX 6000 Ada

Recap: the problem

- A model split over d GPUs idles: backprop is a chain of locks
- **Fix 1, DDG:** use *stale* error gradients so all GPUs back-propagate at once
- **Fix 2, pipelining:** keep many microbatches in flight
  - weights change while microbatches are mid-flight
  - PipeDream *stashes* up to d weight copies per GPU
- **PipeDream-2BW:** only **2 copies** (current + **shadow**), update every m microbatches

$$W^{(t+1)} = W^{(t)} - \nu \nabla f\left(W^{(t-1)}\right)$$

every microbatch uses **1-step-old** weights



Batch 2 (microbatches 5–8) still runs on $W^{(0)}$ although $W^{(1)}$ exists: that is what the shadow buys.

Recap: what we proposed, and the plan we executed

Idea: let **layer freezing** (PolicyFRZ) decide which layers need the shadow

- per layer, per window: gradient variation **GV** and magnitude **AM**
- still changing → keep shadow; barely changing → **drop shadow**; stopped → **freeze**
- hoped for: less memory, same accuracy

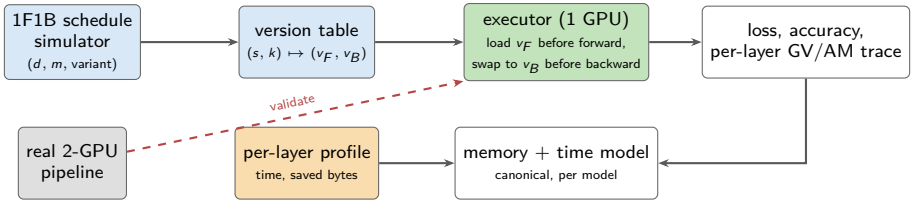
Feedback we built in

- re-run baselines with modern recipes
- a BERT size ladder
- compare each parent on *its own* models
- ablations for every component

Track	Models	Runs
A: DDG re-baseline	ResNet-56/110, CIFAR-10/100	17
B: vs. PolicyFRZ	ResNet-50, CIFAR-10, 160 ep.	18
C: vs. 2BW	BERT Mini, Small, Medium, Base (pre-training) + Base/Large fine-tuning	31 + 36 fine-tunes
LR sweeps	BERT, all sizes	31

$\approx$ 44 h on 2 GPUs. Every pipeline variant on every model, with matched hyperparameters.

Method: exact pipeline emulation on 2 GPUs



- Which weight version a stage uses depends only on $(d, m,$ schedule$)$, not on the GPU count
- Mismatch reproduced by an in-place swap between forward and backward (fp32/TF32; autocast would hide it)

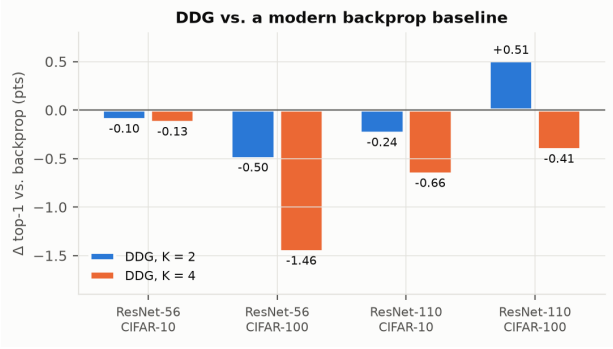
Validation	max difference
V1: vanilla = gradient accumulation	**0**
V2: 2BW = closed-form update	**6e-17**
V3: real 2-GPU pipeline (Flush, 2BW, 1BW)	**0** (bit-exact)
V3: memory, 2BW − vanilla, per stage	**model = measured**

Method: what we compared

	Variant	Weight copies	What it isolates
0	Vanilla (= PipeDream-Flush / GPipe)	1	no staleness; pays pipeline flushes
2	2BW	2	uniform 1-step delay
3	**1BW** = 2BW *without* the shadow	1	delay 0, but some forward/backward mismatch
1	FRZ only (PolicyFRZ threshold rule)	1	freezing alone
4	2BW-Sel: drop shadow where stable	1–2	selective shadow
5	2BW + FRZ	≤ 2	freezing inside a pipeline
6	**Full (ours)**: Sel + FRZ	≤ 2	the proposed merge
7–9	random units / FreezeOut schedule / Adam reset		is the signal needed?

- Freezing rule (PolicyFRZ Eq. 20): freeze unit if $\mathrm{GV} < \theta_1 \wedge \mathrm{AM}/\mathrm{AM}_{\mathrm{ref}} < \theta_2$; decided only at weight-version boundaries; frozen units re-measured every 4 windows
- Thresholds **calibrated on the vanilla runs before** any freezing run; every variant uses the vanilla hyperparameters

Track A: DDG against a modern backprop baseline

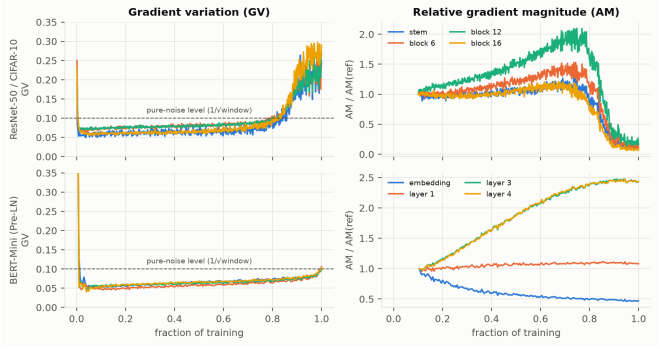


- Our modern BP: ResNet-56/C10 **94.22%** (public recipe† 94.37; DDG paper 93.12)
- With DDG's own recipe (lr 0.01) we get **92.81**: the weak baseline is **entirely the recipe**
- **K=2** still matches BP on CIFAR-10, but -0.5 on CIFAR-100 (2 seeds)
- **K=4** is **below BP everywhere**: -0.13 to -1.46

Takeaway: staleness that is invisible at lr 0.01 and a weak baseline shows up at lr 0.1.

† Modern BP: public ResNet-56 CIFAR recipe, github.com/chenyaofo/pytorch-cifar-models

Finding 1: PolicyFRZ's signals do not single out converged layers



- **GV** sits at or below the noise level for *every* layer after the first few % of training
- **ResNet**: AM rises while GV is low; when AM collapses (lr $\to$ 0), GV rises: "GV low *and* AM low" never holds mid-training
- **BERT**: AM is depth-ordered (embedding settles first), but middle layers' gradients *grow*

Takeaway: the rule can only fire in the low-learning-rate tail, or by forcing thresholds.

Finding 2: a 1-step delay shrinks the stable learning rate

Post-LN BERT-Mini, lr 2e-3 (vanilla's best), step 6,500:

	train loss
Vanilla	**3.83**
2BW	**7.24** (stuck on plateau)
1BW	**7.24** (stuck on plateau)

Any delay prevents leaving the pre-training plateau at rates where vanilla trains.

Pre-LN (what 2BW used, via Megatron), loss at step 5k:

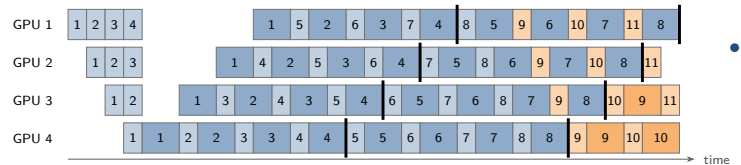
Size	lr	Vanilla	2BW
Mini	1e-3	4.07	4.46
Mini	2e-3	4.07	**6.99**
Small	5e-4	3.65	4.02
Small	1e-3	3.66	**6.79**

Switched Track C to Pre-LN; learning rate = largest grid value where 2BW still trains.

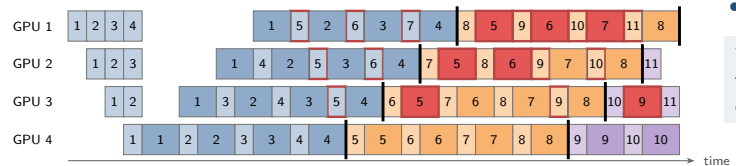
Takeaway: 2BW's "reuse vanilla's hyperparameters" holds only away from the stability edge; the delay roughly halves the usable learning rate.

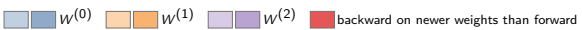
Finding 3: the shadow is what makes 2BW stale

2BW: whole next batch on the old version



1BW: update applied in place, no shadow





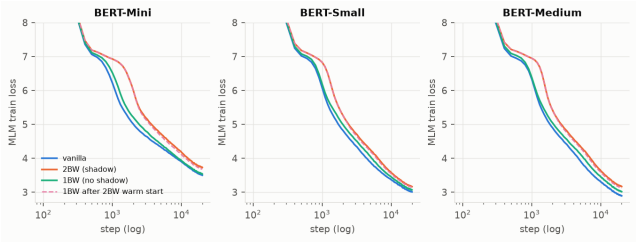
- With the shadow, **every** microbatch is 1 step stale
- Without it, only microbatches in flight at an update mismatch:

$$\text{fraction} = \frac{d - s}{m}$$

- everything else is **fresh**
- one fewer weight copy, too

Takeaway: removing the shadow trades a uniform delay for a small, local mismatch.

Finding 3, evidence on BERT: 1BW closes most of 2BW's gap



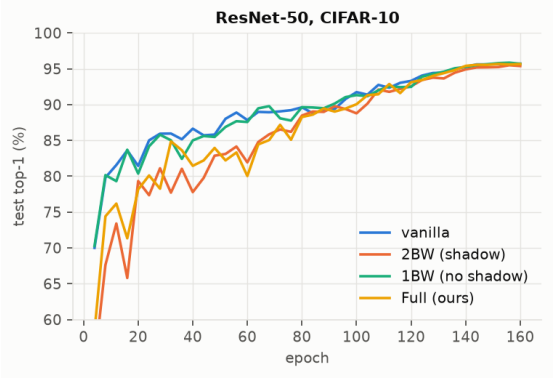
MLM loss	2BW	**1BW**	gap closed
Mini	+0.24	**+0.05**	78%
Small	+0.15	**+0.05**	64%
Medium	+0.23	**+0.12**	50%

Δ vs. vanilla, final validation loss, $d{=}m{=}4$

- 2BW leaves the ∼7.0 plateau ∼500–1,000 steps late
- **2BW for only the first 10%**, then 1BW: gap stays (+0.13 to +0.22)

Takeaway: the first ∼2k steps decide the outcome.

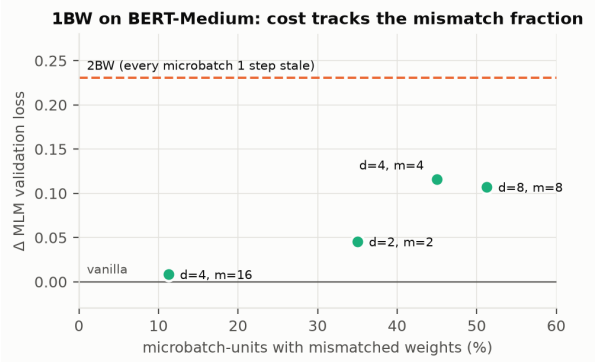
Finding 3, evidence on ResNet-50: 1BW matches vanilla



CIFAR-10, 160 ep.	epoch 4	final
Vanilla (2 seeds)	70.0	95.62 ± 0.09
2BW (2 seeds)	**53.8**	95.39 ± 0.01
1BW (2 seeds)	**70.3**	**95.84 ± 0.14**

- 2BW loses 16 points early and −0.23 at the end, very consistently
- 1BW tracks vanilla from epoch 1
- Memory: dropping the shadow saves little on ResNet-50 (weights ≪ activations); on BERT **5–8%** of stage-1 memory

1BW's cost tracks the mismatch fraction



- BERT-Medium, fixed global batch 128; only d and m change
- Mismatch matches the closed form: 35% / 45% / 51% for $d = 2/4/8$ at $m{=}d$
- $m/d = 4$: 11% mismatched, gap **+0.01**, i.e. **vanilla quality**
- 2BW is independent of d, as its semantics predict (2.97 at $d{=}8$ vs. 2.98–3.02 at $d{=}4$)

Takeaway: more microbatches per batch makes the no-shadow pipeline essentially exact.

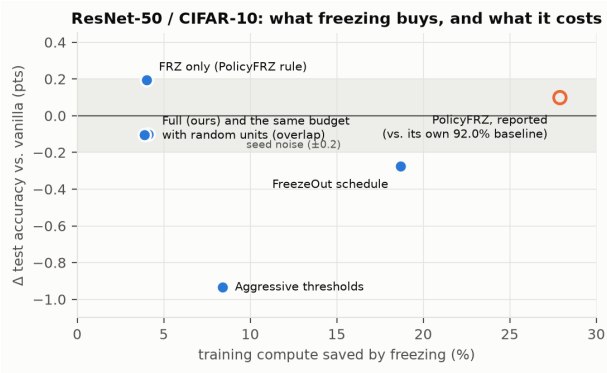
Finding 4: the proposed merge adds little; ablations say why

		ResNet-50 acc. (pts)	BERT-Med. loss
2	2BW (reference)	95.39	3.001
3	**1BW**	+0.45	−0.115
4	2BW-Sel	−0.25	+0.025
5	2BW + FRZ	−0.05	+0.054
6	**Full (ours)**	+0.13	+0.028
7	Full, random units	+0.12	+0.036
8	Full, FreezeOut sched.	−0.05	+0.141
9	Full, Adam reset	−0.07	+0.055

Δ relative to 2BW; ResNet single seeds are ±0.2 noise; BERT ±0.03

- **Removing all shadows** carries the convergence gain
- **Selective** shadow keeps shadows early, exactly when they hurt
- **Freezing** carries the memory/time gain; its cost grows the earlier it fires
- **Signal ≈ random units** at the same budget
- Resetting optimizer state on unfreeze: slightly worse

Finding 5: freezing vs. PolicyFRZ, on PolicyFRZ's own model



- Our ResNet-50 baseline **95.62%** vs. PolicyFRZ's published **92.0%**
- The PolicyFRZ rule costs nothing (95.81) but saves only ∼**4%** of compute, not ∼28%
- Forcing earlier freezing: −0.94 for 8%
- A fixed FreezeOut schedule: 18.7% for −0.28 and −46% stage-1 memory

Takeaway: against a strong baseline, "converged layers" are not visible in GV/AM during pre-training.

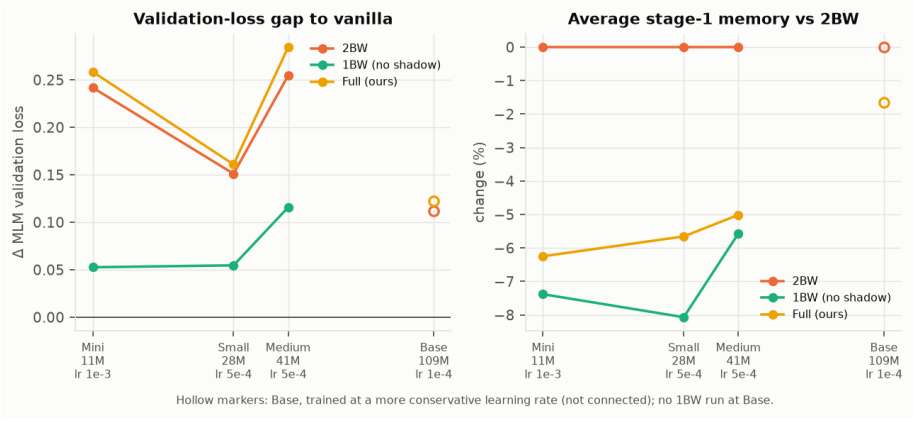
Where freezing does help: fine-tuning converged checkpoints

SST-2 dev acc., 1 epoch, $d{=}m{=}4$	Vanilla	2BW	1BW	**Full (ours)**	Full: frozen / no-shadow
`bert-base-uncased`	92.32	92.55	91.97	91.51	26% / 47%
`bert-large-uncased`	93.35	93.81	–	**94.38**	24% / 36%

- At fine-tuning learning rates the 1-step delay is **harmless** (2BW $\geq$ vanilla)
- Gradients *shrink* as the task is learned, so the rule **fires a lot**: 24–26% frozen
- BERT-Large: -15.6% average stage-1 memory vs. 2BW (Base: -12.9%)
- Accuracy within noise (one seed; 872-example dev set)

Takeaway: the merge behaves as hoped in the regime PolicyFRZ was designed for (fine-tuning), not in pre-training.

Scaling across BERT sizes



- Small → Medium (same lr): deeper model, **larger** 2BW gap (+0.15 → +0.23)
- 1BW's advantage shrinks with depth at fixed m/d (more stages, more mismatch); restored by larger m/d
- Shadow savings: **5–8%** of stage-1 memory at every size
- Base used a conservative lr (1e-4): smaller gap, but the least-trained model

Answers to our five questions

Question	Answer
1. Does DDG match backprop against a modern baseline?	**Partly.** K=2 on CIFAR-10 yes; K=2 on CIFAR-100 -0.5; K=4 loses everywhere (up to -1.46).
2. Does BERT need the safety copy?	**No.** Dropping it closes 50–78% of 2BW's gap (100% at m/d=4), and 100% on ResNet-50.
3. Does the merge beat PolicyFRZ and 2BW on their models?	**Not in pre-training** (within noise of 2BW; $\sim$4% compute saved vs. 28% reported). **Yes in fine-tuning** (-16% memory on BERT-Large).
4. Which component gives which gain? Is GV/AM better than random?	No-shadow $\rightarrow$ accuracy; freezing $\rightarrow$ memory/time. **Signal $\approx$ random.**
5. How do gains scale (Mini $\rightarrow$ Large)?	2BW gap grows with depth at a fixed lr; 1BW recovers most of it (more with larger m/d); freezing pays off only when fine-tuning.

What went wrong, how we caught it, and limitations

Setbacks (all fixed and logged)

- ResNet-50 diverged at lr 0.1 $\rightarrow$ zero-init residual blocks
- Post-LN BERT stuck on the plateau under 2BW $\rightarrow$ Pre-LN (Megatron)
- A short sweep proxy misled us (compressed schedule) $\rightarrow$ real schedule, stopped early
- 1BW silently kept its shadow for 10% of training $\rightarrow$ caught by the cost-model summary; kept as an extra condition
- Out-of-memory crashes, per-run partition drift $\rightarrow$ memory-aware queue, canonical partition

Limitations

- Memory/time for $d > 2$ *modelled* (deltas validated on real 2-GPU)
- Mostly 1 seed (2 for key rows); small effects not interpreted
- 20k-step pre-training = early regime only
- Learning rates differ across BERT sizes
- Cut for compute: CIFAR-100 for Track B, some seeds

Takeaways

1. **Do we need the shadow? No.** It forces a uniform 1-step delay; removing it is cheaper *and* trains better. Use $m/d \geq 4$ and it is essentially exact.
2. **Delays hurt most at the start** and shrink the stable learning rate: protecting the first few thousand steps matters more than which layers keep a copy.
3. **PolicyFRZ's gradient statistics** did not identify converged layers during pre-training against strong baselines; they work when fine-tuning.
4. **Re-baselining matters**: DDG and PolicyFRZ were evaluated against baselines 1–4 points weaker than today's.

Questions?

Full lab notebook: `analysis.md` · code and raw results: `code_repo/`

Backup: Track B full results (ResNet-50 / CIFAR-10)

	Row	Test acc.	Δ vanilla	Avg stage-1 mem vs. 2BW	Rebal. time vs. 2BW	Compute saved
0	Vanilla (2 seeds)	95.62 ± 0.09	–	−0.1%	+66%	–
1	FRZ only	95.81	+0.19	−9.0%	+60%	4.0%
2	2BW (2 seeds)	95.39 ± 0.01	−0.23	–	–	–
3	1BW (2 seeds)	95.84 ± 0.14	+0.22	−0.1%	0%	–
3b	1BW after 10% 2BW warm start	95.78	+0.16	−0.1%	0%	–
3c	1BW, d=2	95.63	+0.01	–	–	–
4	2BW-Sel	95.14	−0.48	0%	0%	–
5	2BW + FRZ	95.34	−0.28	−8.5%	−4.4%	–
6	Full (2 seeds)	95.52 ± 0.14	−0.10	−8.6%	−4.0%	3.6–4.6%
7	Full, random units	95.51	−0.11	−8.5%	−3.9%	3.9%
8	Full, FreezeOut schedule	95.34	−0.28	−46.3%	−18.6%	18.7%
9	Full, Adam reset	95.32	−0.30	−8.8%	−4.2%	–
–	Full, aggressive thresholds	94.68	−0.94	–	–	8.4%
–	Plain (batch 128, no microbatching)	95.39	−0.23	–	–	–

Vanilla = PipeDream-Flush semantics; its +66% time per batch is the flush bubble. Memory and time from the canonical cost model.

Backup: Track C full ladder and Medium ablations

Size (lr)	Van.	2BW	1BW	1BW warm	Full
Mini (1e-3)	3.250	3.492	3.303	3.437	3.509
Small (5e-4)	2.859	3.010	2.914	2.991	3.020
Medium (5e-4)	2.770	3.001	2.886	2.991	3.029
Base (1e-4)	2.982	3.093	–	–	3.104

Final MLM validation loss; Medium vanilla/2BW/full are 2-seed means

BERT-Medium	loss	frozen / no-sh.
FRZ only (no pipeline)	2.811	5% / –
2BW-Sel	3.026	0 / 7%
2BW + FRZ	3.055	4% / 0
Full, random	3.037	4% / 2%
Full, FreezeOut	3.142	25% / 2%
Full, Adam reset	3.056	4% / 2%

SST-2 downstream (same fine-tuning for every checkpoint; ± 0.5 noise): Medium vanilla 86.1, 1BW 85.1, full 85.2, 2BW 84.7; Mini vanilla 84.6, 1BW 83.5, 2BW 82.5, full 82.3.

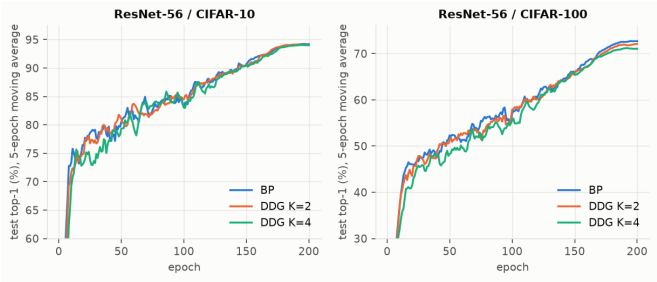
Backup: freezing thresholds and how they were calibrated

- Window 100 optimizer steps; no decisions in the first 10%; frozen units thaw for 1 window every 4; head never frozen
- Thresholds set from offline simulation on the **full vanilla traces** (both seeds agree), before any freezing run:

	ResNet-50 (Track B)	BERT (Track C)
Freeze	$GV < 0.3 \wedge AM_{rel} < 0.5$	$GV < 0.3 \wedge AM_{rel} < 0.6$
Drop shadow	$AM_{rel} < 0.8$	$AM_{rel} < 0.8$
Expected budget	$\sim$13% of unit-windows, from 84% of training	$\sim$10–12%, embedding unit, from 37–52%
Stress test	$GV < 0.073$ (p25), AM ignored	–

- Shadow rule uses AM, not GV: the 1-step error scales with the *update size*; GV is highest exactly when updates are smallest

Backup: DDG training curves, and the paper-recipe reproduction



ResNet-56/C10	ours	paper
BP, paper recipe	92.81	93.12
DDG K=2, paper recipe	92.70	93.11
BP, modern recipe	94.22	–

Gaps open late, as the learning rate anneals. Exact DDG (Eqs. 6–9): gradients at the weights each module's forward pass used; K=1 = backprop, same code path.